Doc. no.:	P0325R2
Date:	2018-07-11
Audience:	LEWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

`to_array` from LFTS with updates

Introduction

This paper proposes to adopt `to_array` from Library Fundamentals TS along with an rvalue-reference overload based on the feedback from the users.

```
auto lv = to_array("foo");               // array<char, 4>
auto rv = to_array<int>({ 'a', 'b' })    // array<int, 2>
```

Discussion

In C++17 we introduced deduction guide on `std::array`:

```
auto a = array{ 1, 2, 3 };
```

But it is impossible to create an `array<char, 4>` from `array{ "foo" }` because otherwise, one won't be able to explain why `array{ "foo", "bar" }` creates `array<char const*, 2>`. So one overload set, despite whether it's `std::array` deduction guide or `make_array` from LFTSv2, is not enough to serve the purpose of "creating a `std::array` from elements" and "creating a `std::array` by copying from a built-in array" at the same time. That is why[1] we introduced `to_array` in LFTSv2:

```
auto a1 = to_array("foo");          // array<char, 4>
auto a2 = array{ "foo" };           // array<char const*, 1>
```

For the same reason, `to_array` should be adopted into the IS.

However, the original version of `to_array` only takes an lvalue reference-to-array, and users reported[2] some issues regarding this design. The first issue is that it becomes very hackish to deduce only the array bound from a *braced-init-list*:

```
auto x = to_array<int const>({ 2, 4 });     // array<int, 2>
```

The language treats *braced-init-list* as a prvalue of an array type when deducing against reference-to-array. The code above adds `const` to coin a reference that can bind to such a prvalue, and leaves the `const` to be stripped later. Anyhow, the code isn't doing what it says.

The second issue is that it does not support move-only elements, such as `unique_ptr`, no matter how you hack.

So we should add the rvalue-reference overload to fill this hole regarding type system.

Impact on the Standard

The following table illustrates the uses and the corresponding overload resolution after adopting the proposed addition. Given

```
int a[3] = {};
struct A { int a; double b; };
```

use	argument	overload	result
`to_array("meow")`	lvalue `char const[5]`	lvalue ref	`array<char, 5>`
`to_array({1, 2})`	prvalue `int[2]`	rvalue ref	`array<int, 2>`
`to_array<long>({1, 2})`	prvalue `long[2]`	rvalue ref	`array<long, 2>`
`to_array(a)`	lvalue `int[3]`	lvalue ref	`array<int, 3>`
`to_array(std::move(a))`	xvalue `int (&&)[3]`	rvalue ref	`array<int, 3>`
`to_array<A>({{3, .1}})`	prvalue `A[1]`	rvalue ref	`array<A, 1>`
`to_array((int[]){3, 4})`	C99 compound literal	see below	`array<int, 2>`

The last entry is a valuable case to study. Compound literals are lvalues in the C standards, but C does not have rvalue objects of compound types anyway. Both Clang and GCC implement C99 compound literals as extensions in C++ and treat them as prvalues. Let's take a closer look at this feature,

```
(int[]){3, 4}
```

It is an array literal formed by a C-style cast notation (like `(char)`) for an array type followed by an initializer list. In the same way, we can interpret `to_array` as

```
to_array<int>({3, 4})
```

a `std::array` literal formed by a C++ style cast notation (like `static_cast<char>(...)`) for the array element type surrounding an initializer list.

An interesting consequence of this proposal is that we no longer need `make_array`:

LFTSv2	C++20
`make_array(1, 2, 3)`	`array{ 1, 2, 3 }`
`make_array<long>(1, 2, 3)`	`to_array<long>({ 1, 2, 3 })`

In the future, we may have `array<long>{ 1, 2, 3 }` to replace this specific use of `to_array`, but that will not defeat the primary purpose of `to_array` – a facility to create `std::array` by copying or moving from a built-in array.

Supplying the bound with `to_array<T, N>({ ... })` is more strict compared to `array<T, N>{ ... }`, because brace elision in aggregate initialization can sometimes give unexpected outcomes:

```
using E = complex<double>;

// Nicol's quiz time
auto a = array<E, 10>{{1, 2}, {3, 4}};
auto b = array<E, 10>{{{1, 2}, {3, 4}}};
auto c = array<E, 10>{{1, 2}};
```

Implementation

Godbolted: https://godbolt.org/g/GqbygN (https://godbolt.org/g/GqbygN)

Wording

This wording is relative to N4762.

New section 21.3.7.6 [array.creation] (between [array.zero] and [array.tuple], which was 21.3.7.6):

21.3.7.6 Array creation functions [array.creation]

```
template <class T, size_t N>
  constexpr array<remove_cv_t<T>, N> to_array(T (&a)[N]);
template <class T, size_t N>
  constexpr array<remove_cv_t<T>, N> to_array(T (&&a)[N]);
```

> *Returns:* For all i, $0 \leq i <$ `N`, an `array<remove_cv_t<T>, N>` initialized with `{ a[`i`]... }` for the first form, or `{ std::move(a[`i`])... }` for the second form.

Add those signatures to 21.3.2 [array.syn]:

```
namespace std {
  // 21.3.7, class template array
  template<class T, size_t N> struct array;

  …
  template<class T, size_t N>
    void swap(array<T, N>& x, array<T, N>& y) noexcept(noexcept(x.swap(y)));

  // 21.3.7.6, Array creation functions
  template <class T, size_t N>
    constexpr array<remove_cv_t<T>, N> to_array(T (&a)[N]);
  template <class T, size_t N>
    constexpr array<remove_cv_t<T>, N> to_array(T (&&a)[N]);
  …
}
```

Acknowledgements

Thank Peter Sommerlad for the inputs on this paper.

References

1. N4391 *make_array, revision 4*. http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4391.html (http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4391.html) ↩
2. LWG 2814 *`to_array` should take rvalue reference as well*. https://cplusplus.github.io/LWG/issue2814 (https://cplusplus.github.io/LWG/issue2814) ↩